AI651 — Deep Learning for Space, Time and Graphs

Assignment 2

Rifa Salman

Fall 2026

Code: https://github.com/RifaSalman/AI651-Assignment-2

Question 1 is a forecasting design and was not trained. Question 2 was trained on an NVIDIA Tesla T4. Learned numbers are mean (seed 0, seed 1). Errors are three significant figures. Values below 10^{-3} are in scientific notation.

Question 1 — River Forecasting Proposal

The service issues a forecast every ten minutes. At issuance time τ it returns the water level one hour later at S1, S2, S3 and S4. The river graph is taken from the project figure. The reservoir releases into the main stem at S1. The monitored tributary, gauged at T1, joins between S1 and S2. The unmonitored tributary joins between S2 and S3. Rain gauges R1 and R2 sit on the right-hand side of the valley, toward that unmonitored catchment and the lower river. S4 is the last station before the downstream town. The town is not a forecast target.

The forecaster is a small directed graph model. Connectivity and distances are fixed inputs. Travel time is not. A lag kernel on each edge learns how long water takes to move, and it is allowed to change that lag when the river is high. A latent node stands in for the unmonitored tributary. The delivered product is the 0.5 quantile at τ+60 min, together with a 0.1–0.9 interval, at each of S1–S4.

1. Architecture diagram

Figure Q1 shows the data sources, the alignment step, the graph, the learned blocks, and the four forecasts. Tensor shapes include the batch axis B.

The nine nodes are Res, S1, T1, S2, U, S3, S4, R1 and R2. U is the unmonitored tributary and has no level sensor. Edges point downstream only:

- Res → S1 → S2 → S3 → S4
- T1 → S2
- U → S3
- R1 → U
- R2 → S3 and R2 → S4

History is aligned onto one 10-minute axis, τ−35 through τ (36 steps, 6 hours). The horizon is τ+1 through τ+6. Node features are stacked to (B, 36, 9, F). A shared GRU reads each node’s own history. Each edge then sends a mixture of lagged upstream states. Four message-passing steps are enough for a release at Res to reach S4, and for rain at R1 to reach S4 through U, inside one update. A six-step rollout follows. The only future input is the reservoir schedule that had already been announced at τ. Readout heads on S1–S4 produce (B, 6, 4, 3): six lead times, four stations, quantiles 0.1, 0.5 and 0.9. The service publishes the slice at lead time 6.

2. Assumptions

Supplied facts, not assumptions: levels at S1–S4 and T1 every ten minutes; rainfall at R1 and R2 with timestamps and locations; recorded releases and future schedules announced before issuance, with announcement and effective times; station locations, connectivity and distances; no precise travel times; no measurement of the unmonitored tributary; no future rainfall and no future water levels at issuance. The figure fixes where the tributaries join.

A1. A 6-hour history is long enough for a disturbance at the reservoir or at R1 either to have reached S4 or to still be inside the window. Assignment 1’s teaser had a disturbance taking 12 samples, two hours, to travel from station 3 to station 4, so six hours covers the longer path from

the dam to S4 with margin. If this is short, a release older than six hours is invisible and the forecast falls back on recent local level. The encoder length is 36 because of A1.

A2. Level records already share a 10-minute clock. Rainfall timestamps are irregular and are summed into those same bins, in millimetres per bin. Gauges are usually stored that way, and one clock is what lets a rain bin line up with a level bin.

A3. At least one year of record is available, covering a range of seasons and releases. A shorter record leaves the quantiles miscalibrated in unseen seasons. The design does not add a separate seasonal model; it adds time-of-day and day-of-year features (A3).

A4. Levels are stage in one vertical datum, in metres, with noise of a few centimetres, small next to a flood rise. There is no bias-correction block.

A5. Travel time shortens as the river rises, and the recent upstream level is a usable stand-in for that stage. The lag weights take distance and upstream level as inputs. A fixed lag is not used.

A6. R1 represents the headwaters of the unmonitored tributary, and R2 represents the lower right-hand valley that drains toward S3 and S4. The figure shows the gauges but not catchment polygons, so this assignment of edges is an assumption. It decides which forecasts rain is allowed to touch: R1 cannot move S1.

A7. Rain over the next hour is unknown and is not invented. That uncertainty is carried by the quantile width. No weather-model input is required for the system to run.

A8. A schedule whose announcement time is at or before τ is known exactly for the future steps. The actual release can differ. Past steps carry both the actual and the schedule; future steps carry the schedule only.

A9. One shared model serves all four stations. Upstream level is an input to the downstream forecast, and the same lag mechanism applies on every reach. Four separate models would drop that sharing.

A10. The authority needs a single level and a range. The 0.5 quantile is the issued level. The 0.1 and 0.9 quantiles are the issued range.

A11. A report usually arrives within ten minutes, but not always. A value is usable at τ only if its timestamp is at or before τ and its arrival time is at or before τ. Anything else is missing, not late-filled after the forecast has gone out.

A12. Channel distance is fixed. Travel time changes; the map does not. Distance is a static edge feature. The lag is learned.

A13. Backwater is negligible at the spacing in the figure, so edges point downstream only. A downstream constriction that holds water up at S3 would not be seen. That case is listed under limitations.

3. Forecasting task and data specification

Issuance time τ is the end of a 10-minute bin, and a forecast is issued at every such bin. The history window is the 36 bins $\tau-35, \ldots, \tau$. The horizon is one hour: lead times of 10, 20, 30, 40, 50 and 60 minutes, written $\tau+1, \ldots, \tau+6$. The required output is the level at S1–S4 at $\tau+6$. The intermediate lead times are trained as well, so the one-hour forecast is the last step of a trajectory rather than a jump.

Information available at τ: levels and rain whose observation time and arrival time are both $\leq \tau$; actual releases already recorded by τ; the release schedule for future effective times, but only the version announced at or before τ; the static graph. Not available: future rain, future levels, the unmonitored inflow, and any schedule that has not been announced yet.

Alignment. Every series is placed on the same 10-minute index. A level stamped at time t is written into the bin that ends at t. Rain is the sum over (t−10 min, t]. Nodes do not share a spatial grid. They are stacked along a node axis in the fixed order Res, S1, T1, S2, U, S3, S4, R1, R2.

Missing entries are 0, with the mask set to 0, so a missing level is not read as a water level of zero. The future schedule is a separate tensor. It is not concatenated into the history.

Series	Shape	Axes	Time coverage
Level history, S1–S4 and T1	(B, 36, 5)	batch, history step, station	τ−35 … τ
Level mask and staleness	(B, 36, 5) each	same	same; staleness in minutes since the last arrived observation
Rain history, R1 and R2	(B, 36, 2)	batch, history step, gauge	millimetres in each bin τ−35 … τ
Rain mask	(B, 36, 2)	same	same
Actual release	(B, 36, 1)	batch, history step, reservoir	τ−35 … τ
Schedule that was in force in the past	(B, 36, 1)	same	the schedule known at that past time
Announced future schedule	(B, 6, 1)	batch, lead time, reservoir	τ+1 … τ+6, announcement time ≤ τ
Edge index	(2, E), E = 9	source node, target node	static
Edge distance	(E,)	one distance per edge	static, along-river
Clock features	(B, 42, 4)	batch, history+horizon, sin/cos of hour and day-of-year	τ−35 … τ+6
Output quantiles	(B, 6, 4, 3)	batch, lead time, station S1–S4, quantile 0.1 / 0.5 / 0.9	τ+1 … τ+6
Issued forecast	(B, 4)	batch, station	τ+6, the 0.5 quantile

4. Model computations and design justification

Let V be the nine nodes and let $z_{v,t}$ be the hidden state of node v at time t. Write $x_{v,t}$ for the observed value (level, rain, or release), $m_{v,t}$ for its mask, and $s_{v,t}$ for staleness. Clock features are c_t. d_{uv} is the along-river distance of edge u→v.

Node encoder, shared across nodes of the same type (level station, rain gauge, reservoir, latent). For a node that has a sensor,

$z_{v,t} = \mathrm{GRU}(z_{v,t-1}, W [x_{v,t} \odot m_{v,t}, m_{v,t}, s_{v,t}, c_t])$.

The mask multiplies the value, so a missing entry contributes nothing through the value channel. U has no level; its input is the zero vector plus the clock, and it receives rain only through messages.

Edge message. For each downstream edge and each lag $\ell = 0, \ldots, 35$,

$\alpha_{uv,\ell} = \mathrm{softmax}_\ell \, \mathrm{MLP}(d_{uv}, z_{u,t})$.

$m_{u\to v,t} = \Sigma_\ell \, \alpha_{uv,\ell} \, z_{u,\, t-\ell}$.

The weights depend on distance and on the current upstream state, which is A5. They sum to one. A long edge can put its mass on a long lag. A high upstream state can move that mass toward shorter lags.

Message passing. Four rounds, with the GRU above run again so each round can mix a new set of incoming messages:

$z_{v,t} \leftarrow \mathrm{GRU}(z_{v,t}, \Sigma_{u:\, u\to v} \, m_{u\to v,t})$.

Four rounds match the longest path, Res to S4, which is four edges. Fewer rounds would make S4 wait several time steps before a release even enters its state, and that wait would be the same for a short reach and a long one. The lag kernel, not the depth, is what represents travel time.

Rollout. For lead $k = 1, \ldots, 6$ there is no new level and no new rain. The reservoir node receives the announced release $r_{\tau+k}$ instead of a level. Other nodes receive only messages:

$z_{v,\tau+k} = \mathrm{GRU}(z_{v,\tau+k-1}$, messages at $\tau+k$, and $r_{\tau+k}$ if v is Res).

Readout, stations $s \in \{S1, S2, S3, S4\}$ and quantiles $q \in \{0.1, 0.5, 0.9\}$:

$\hat{y}_{s,\tau+k}^{(q)} = w_q \cdot z_{s,\tau+k} + b_q$.

Training objective. On historical issuances the future levels are known after the fact, so they can

be targets. They are never inputs. The pinball loss for quantile q is $\rho_q(e) = q\,e$ if $e \geq 0$, and $(q-1)\,e$ otherwise, with $e = y - \hat{y}$. The objective is the mean of ρ_q over the batch, the four stations, the six leads and the three quantiles. The 0.5 term trains the issued level. The 0.1 and 0.9 terms train the range. No squared-error term is added: the range is the part the flood desk uses, and pinball trains it directly.

What is physical, and what is learned. The edge list, the downstream direction and the distances are taken from the river. They are not learned. The map from those inputs to a level is learned: the GRU weights, the lag MLP, and the readout. Mass conservation is not enforced. The unmonitored tributary and the unknown rating curves make a hard balance equation under-determined, so a violated balance would be fitted by distorting U. The directed graph is the physical constraint that remains: S1 does not depend on S4, T1 does not depend on S3, and rain does not flow upstream.

How each driver reaches each forecast.

- Reservoir. The future schedule enters only at Res. From there it crosses S1, S2, S3 and S4. A release announced at τ can change S1 inside the hour. It changes S4 inside the hour only if the learned lag is shorter than an hour. If the lag is longer, S4's one-hour forecast moves only when the release is already in the history.
- Upstream main-stem level. S1's state is an input to S2, S3 and S4. S4's state is not an input to S1 (A13).
- Monitored tributary. T1 sends a message into S2, and from S2 onward to S3 and S4. It does not enter S1.
- Rain. R1 messages U, and U messages S3, so R1 affects S3 and S4 only. R2 messages S3 and S4 directly. Neither gauge touches S1 or S2. That is A6.
- Unmonitored tributary. U has no level. Its state is whatever the rain messages and the downstream residual leave it. S3 and S4 can rise when S2 does not, and that rise is attributed to U during training.

The one-hour product uses lead $k = 6$. Leads 1–5 are kept in the loss so the lag has a target at every step, not only at 60 minutes.

5. Operation and exceptional conditions

(i) Missing observations. The value is set to 0 and the mask to 0. Staleness counts the minutes since the last observation that did arrive. The GRU sees the mask and the staleness, so it can hold the last reliable state instead of treating a gap as a level of zero. Gaps are not filled with an interpolated level before the model runs. An interpolated value would look like a measurement.

(ii) Delayed observations. At issuance, any record whose arrival time is after τ is dropped, even if its sensor timestamp is earlier. It is a missing value for this forecast. When it arrives, a later issuance can use it inside the history window. A forecast already issued is not revised in this design. Revising it would break the ten-minute product the authority asked for.

(iii) Announced future releases. If the announcement time is ≤ τ, the scheduled release on τ+1, …, τ+6 is the future reservoir input. Schedules announced later are ignored until the next issuance. The effective time selects the lead step. A schedule that takes effect after τ+6 is outside this horizon and is not an input. During training, each past issuance is rebuilt with only the schedule that had been announced by that issuance, so the model does not train on releases it could not have known.

(iv) Scheduled versus actual releases. On past steps the actual release and the schedule then in force are both inputs, and so is their difference. On future steps only the schedule exists. A gate that opens late or only part way is something the model has seen as a residual in the history. The residual on the future step is not observed, so it widens the quantile range rather than shifting the input. If actuals are usually below the schedule, that pattern is in the training pairs and the 0.5 quantile can learn it. A one-off override that has no history still lands in the range, not in the point forecast.

Feasibility every ten minutes. The graph has 9 nodes and 9 edges. One forecast is a 36-step GRU, four message-passing rounds and a 6-step rollout, with a hidden width of a few tens. That is a fraction of a second on one CPU core. The ten-minute budget is set by when the gauges report, not by the forward pass. Training is offline, on stored issuances. The online step is the forward pass only. No benchmark is claimed beyond this size argument.

6. Uncertainty, limitations, and additional information

Unmonitored tributary. A storm on that branch raises S3 and S4 with no level gauge on the branch. The response is the latent node U, driven by R1 and R2, and a wider interval when rain is falling and the rise from S2 and T1 does not account for the rise at S3. What remains unresolved is rain that misses both gauges. The forecast is then late: it moves only after S3 itself has risen. Request: a level or flow gauge on the unmonitored tributary, at the confluence. It would turn U into an observed node and shorten the warning. The system still runs without it. Floods from that branch will be the weak case.

Changing travel times. A fixed delay would be early at high stage and late at low stage. The response is A5: the lag mixture depends on upstream level as well as distance. What remains unresolved is a change that level does not represent, such as ice, a debris dam, or a new cut. Request: a small set of travel-time measurements at low and high stage, or a velocity sensor at S2 and S3. They would check the learned lags and give a starting point for an unseen stage. The system still runs without them. Lags are learned from the historical record. An extreme stage outside that record will be off.

No future rainfall. Rain that starts after τ can reach a nearby station inside the hour. The response is to leave it out of the inputs and let the 0.1–0.9 interval carry it. The point forecast will not rise until a gauge has seen the rain. Request: a one-hour rainfall forecast, from radar or a numerical weather model, as an extra future input on R1 and R2, with its own uncertainty. The system still runs without it. Intervals stay wider, and a storm that begins after issuance is missed until it hits a gauge.

Schedule against actual. An unannounced gate change is invisible until the actual release is recorded. The response is the past residual feature and the quantile range. Request: gate telemetry within a minute of the movement, written into the actual-release series. The system still runs without it when operators stay close to the announced schedule.

Sensor datum and rating. A drifting zero looks like a slow change in level. Request: calibration dates and, if operations are ever done in discharge, the rating curves. The system still runs without them and forecasts the level as reported. It does not convert level to flow.

Backwater and the town. A13 drops upstream edges. If a bridge or a constriction downstream of S4 holds water, S4 can rise for a reason that is not in the graph. Request: a note of control structures and their operating rules. The system still runs without them. This error is local to the affected station.

The forecast remains usable for the four stations with the data already offered. The extra requests improve the unmonitored branch, the travel-time check, and the next-hour rain. None of them is required for a forecast to be issued every ten minutes.

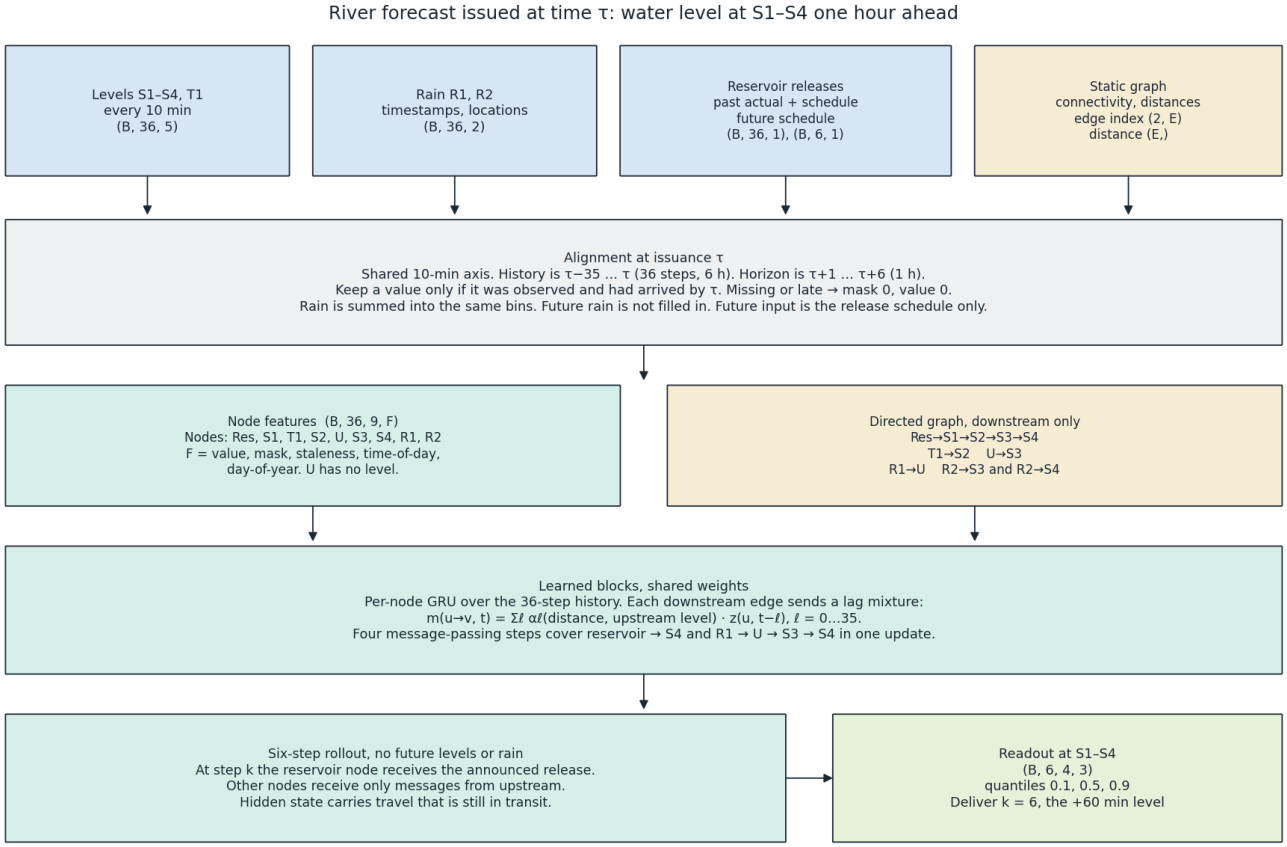


Figure Q1. Forecasting architecture. Blue boxes are observations, sand boxes are fixed by the river, green boxes are learned, and the issued product is the +60 min level at S1–S4.

Question 2 — Motion or Memory?

Hardware for every learned model is an NVIDIA Tesla T4 on Kaggle. Training time is the mean wall-clock time per epoch over 30 epochs, excluding the dataset download. Inference time is the wall-clock time to score all 1,000 original test videos at batch size 32, after each batch is on the device, under model.eval() and torch.no_grad(). Seeds 0 and 1 are reported as mean (seed 0, seed 1). Figures and GIFs use seed 0. Model A took 11.9 s (11.9 s, 11.9 s) per epoch.

Table 1. Transformation and window checks

Check	Expected	Result
(a) max |M(M(x)) − x|	0	0, torch.equal True
(b) max |R^4(x) − x|	0	0, torch.equal True
(c) max |M(R(M(x))) − R^{−1}(x)|	0	0, torch.equal True
(d) bright pixel of M(p)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target	1–5 / 6–10	1–5 / 6–10
(g) 2× input / target	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	same
(h) 3× input / target	1, 4, 7, 10, 13 / 16, 19	same

Table 1. Checks (a)–(h) on the functions used in the experiments. x is torch.rand(20, 1, 64, 64) after torch.manual_seed(0). p is zero except p[7, 0, 10, 20] = 1. Model A has 480,784 parameters.

Figure 1. Optical flow on the first 50 training videos (103050 pixels kept, median speed 3.32 px/frame).

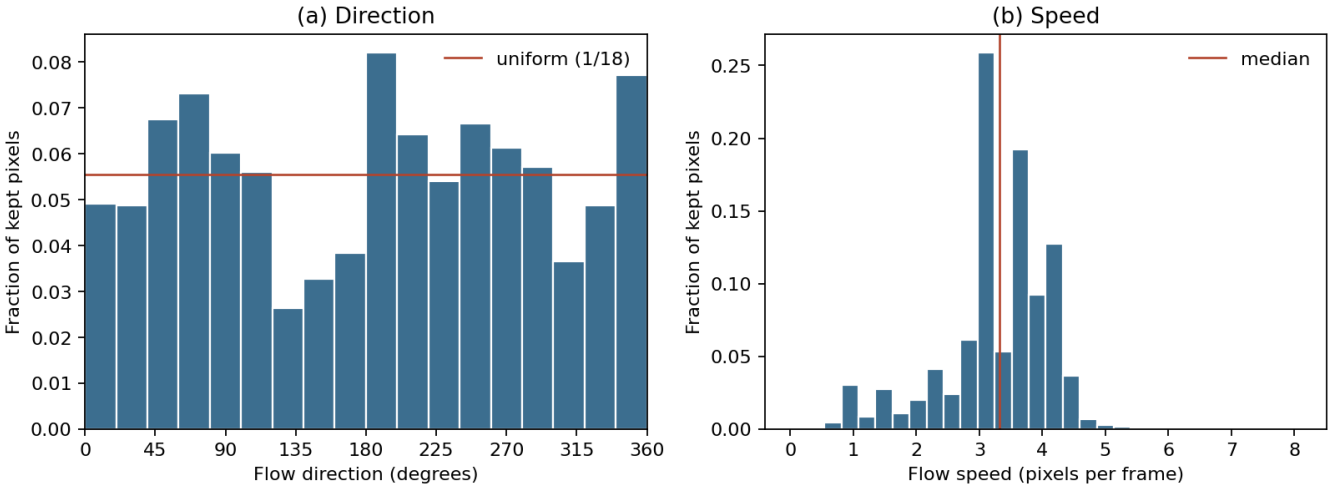


Figure 1. Optical flow on the first 50 training videos, 950 frame pairs, Farneback settings as specified. Pixels kept: intensity above 0.2 on both frames and magnitude above 0.5. Kept pixels: 103050. Median speed 3.32 pixels per frame. The horizontal line in (a) is 1/18.

Q2.1

Mirrored and rotated trajectories are partly familiar, and they are not exchangeable with the training distribution. Every 20° bin in Figure 1(a) is occupied, so a reflected or rotated direction is not a new kind of motion. The bins are not flat: the fullest holds 0.082 of the kept pixels and the emptiest 0.026, a ratio of 3.11, against a uniform level of 1/18 ≈ 0.0556. Reflecting every direction moves about 0.13 of the probability mass (total variation between the histogram and its mirror). A 90° rotation is not aligned to the 20° bins; the closest shifts, 4 and 5 bins, move about 0.18 of the mass. A typical speed is the median, 3.32 pixels per frame (mean 3.27, 90th percentile 4.12). Twice that median is 6.63 pixels per frame. Three times it is 9.95. Twice the 90th percentile is 8.25. Optical flow is unreliable where digits overlap. The kept set already drops pixels that are dark in either frame, but overlap boundaries can still invent vectors and fatten the tails. That bias would have to move the median a long way to change the comparison with an 8-pixel reach. It does not: even a median of 4 would put 2× on the reach and 3× beyond it.

Q2.2

A mirror changes both the appearance and the motion statistics. A mirrored digit is a different

shape, and the direction histogram is reflected. A 90° rotation changes appearance more than a mirror does, and it sends every direction into a different part of Figure 1(a). A speed-up does not change the digit. It only increases the displacement between the frames the model sees. The three therefore separate appearance, an unfamiliar direction, and the size of the step. Constant-velocity motion with elastic bounces is still valid motion after all three, so a model that had learned the motion rule should cope with all of them.

Table 2. Baseline forecast error

Model	MSE1	MSE2	MSE3	MSE4	MSE5	E(5)	Parameters
A	0.0171 (0.0173, 0.0168)	0.0205 (0.0209, 0.0201)	0.0228 (0.0233, 0.0223)	0.0246 (0.0251, 0.0241)	0.0263 (0.0268, 0.0259)	0.0223 (0.0227, 0.0218)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Table 2. Test-set MSE by horizon at 1×, t = 1. Learned entries are mean (seed 0, seed 1).

Figure 2. Seed-0 model A on three test videos. Rows are input, prediction, and ground truth.



Figure 2. Seed-0 model A on test videos 9000, 9007 and 9012. Each block is input, prediction, ground truth, five frames each. Video 9000 is the overlapping pair (a 5 and a 6 merge and separate). Videos 9007 and 9012 keep two components through frames 1–10.

Q2.3

pixel_unshuffle(4) makes each cell of the 16×16 grid stand for 4 original pixels, centre to centre. The 4×4 patch width is not added. The 1×1 head, the unshuffle and the shuffle do not move information. Each ConvLSTM layer is a 3×3 convolution with dilation 1, so it reads offsets −1, 0 and +1 and adds a Chebyshev radius of one cell. In one step the new input passes through layer 1 and then layer 2. Layer 1's previous hidden state takes that same path. Layer 2's previous state passes through layer 2 only, which is shorter. The longest path is 2 cells, 8 original pixels along a row or a column. The corner of that neighbourhood is $8\sqrt{2} \approx 11.3$ pixels. The reach reported here is the axis-aligned one, 8 pixels.

Q2.4

Model A's E(5) is 0.0223, against 0.0426 for blank and 0.0641 for copy-last. It is clearly better than both references on ordinary 1× clips. The error grows with the horizon, from MSE1 = 0.0171 to MSE5 = 0.0263. That is the closed-loop rollout: each later frame is conditioned on the network's own previous prediction, so blur accumulates. In Figure 2 the separated digits in videos 9007 and 9012 stay recognisable and follow the right general direction, while becoming softer. Video 9000 is the overlap. The 5 and the 6 merge, and the forecast turns them into one soft blob instead of recovering two digits the way the ground truth does. Overlap is where the model loses which digit is which, not only where it is.

Q2.5

Q2.5 (written before any mirror, rotation, or speed experiment)

Model A will fail the mirror test and the rotation test. It is trained only on ordinary 1× clips, and a plain ConvLSTM has no mirror or 90° symmetry built in. Figure 1(a) is not flat: every 20° bin is occupied, so a reflected or rotated direction is not a completely new kind of motion, but the fullest bin is about 3.1 times the emptiest. Mirroring the directions moves about 13% of the probability mass, and a 90° rotation moves about 18%. I expect the network to tie its forecast to the digit shapes and to the directions it saw most often. Rotation should be worse than mirroring, because a 90° turn changes what a digit looks like more, and it sends every direction into a different part of the histogram.

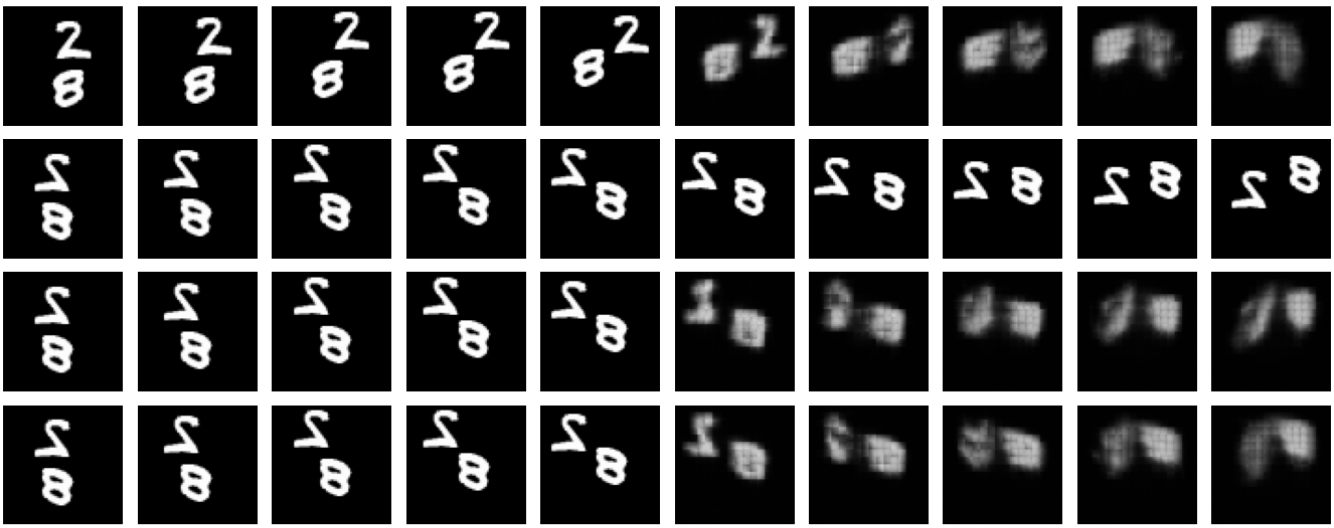
On speed, model A should mostly keep up at 2× and break at 3×. One step reaches 8 original pixels: the input passes through two 3×3 convolutions on the 16×16 grid, each grid step is 4 original pixels, and the 4×4 patch width is not added. Figure 1(b) has median speed 3.32 pixels per frame and a 90th percentile of 4.12. Twice the median is 6.64, still inside 8 pixels. Three times the median is 9.95, which is beyond an axis-aligned step of 8. The fast tail at 2× is about 8.2 pixels per frame, so a few 2× clips may already smear, but the clear breakdown should be at 3×.

Table 3. Mirror and rotation

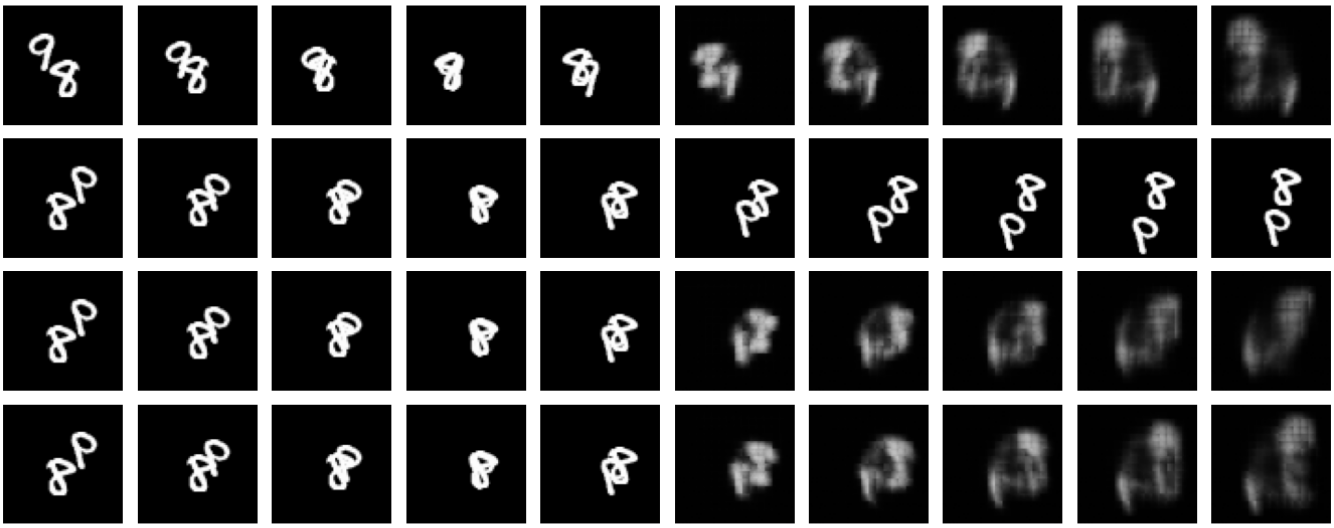
Model	E(5)	C_M	E_M	C_R	E_R
A	0.0223 (0.0227, 0.0218)	0.00174 (0.00145, 0.00203)	0.0229 (0.0232, 0.0225)	0.00293 (0.00237, 0.00349)	0.0239 (0.0241, 0.0238)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

Table 3. Consistency gaps and transformed error at 1×, t = 1. Blank and copy-last are exactly consistent. Their C_M and C_R are 0.

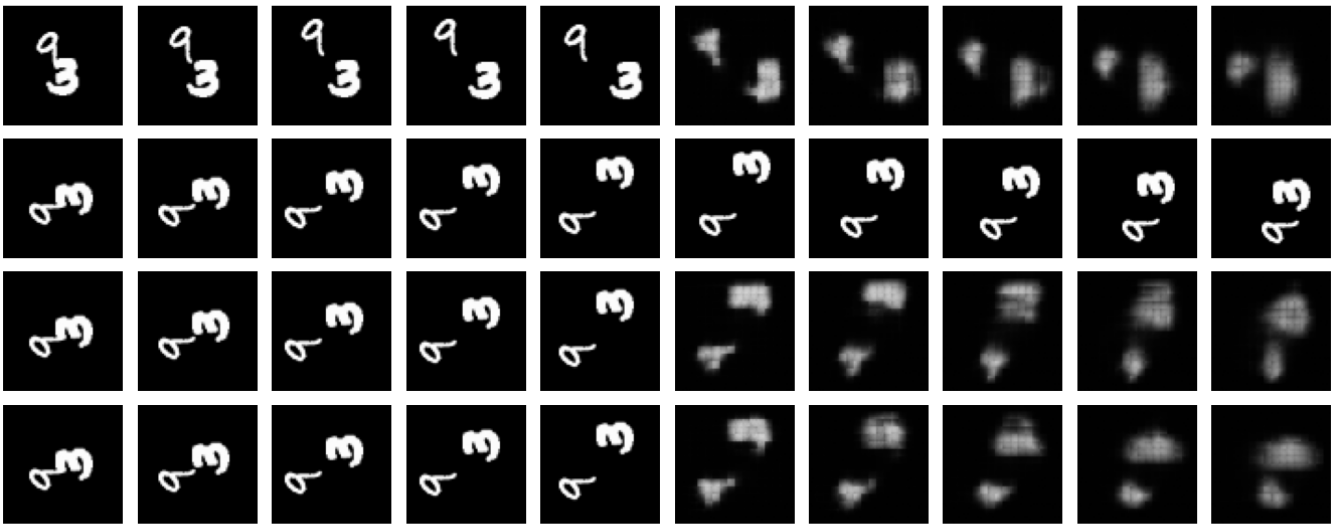
mirror video 9642



mirror video 9279



rotation video 9168



rotation video 9925

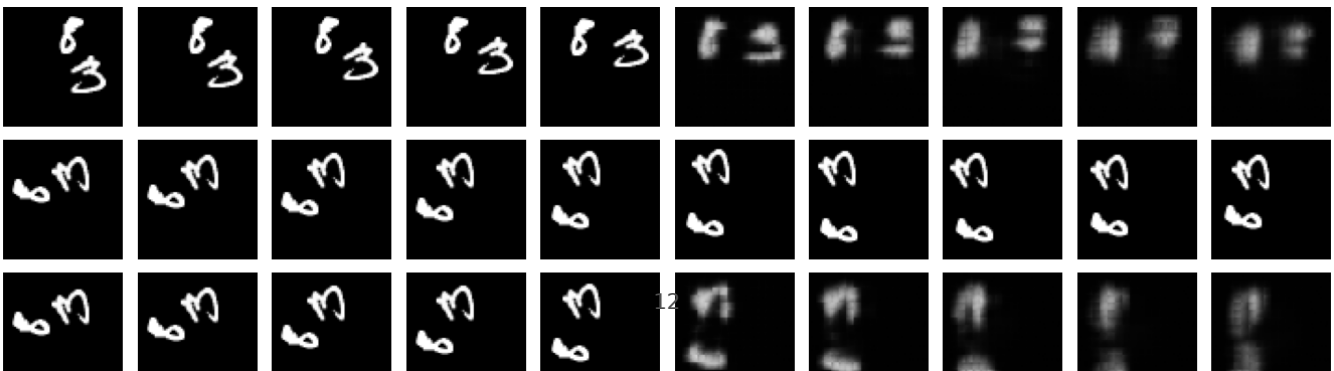


Figure 3. Seed-0 model A, the two test videos with the largest per-video C_M (9642, 9279) and the two with the largest per-video C_R (9168, 9925). Rows in each panel are X | f(X); T(X) | T(Y); T(X) | f(T(X)); T(X) | T(f(X)). GIFs, one horizon at a time: Question2/outputs/gifs/figure3_mirror_0.gif, figure3_mirror_1.gif, figure3_rotation_0.gif, figure3_rotation_1.gif.

Figure 4. Seed-0 model A consistency gaps on the test set. Marked videos are those in Figure 3.

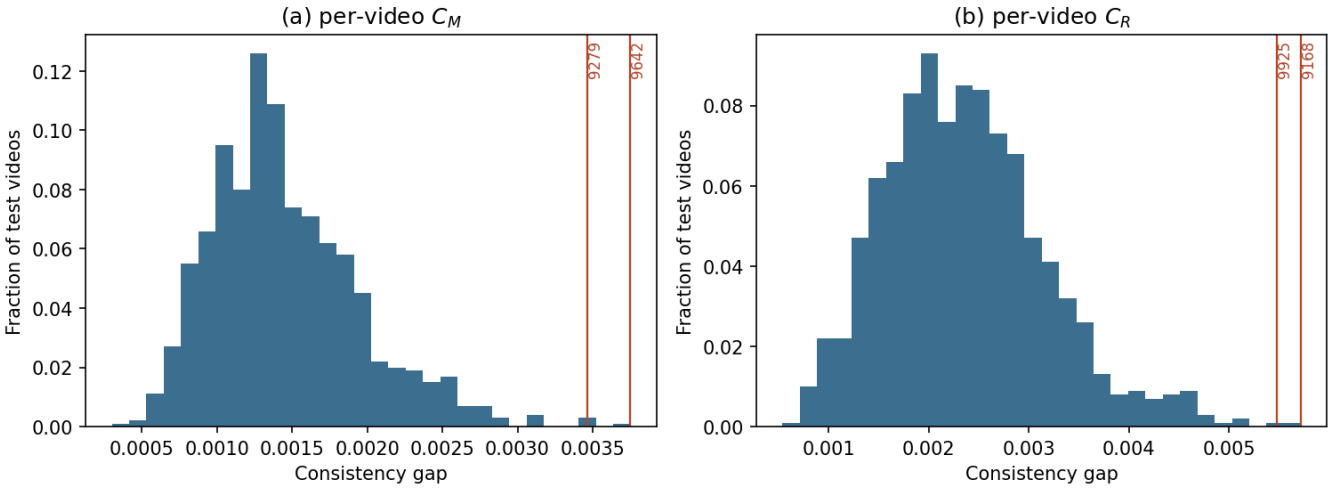


Figure 4. Per-video C_M and C_R of seed-0 model A on the 1,000 test videos. Red lines mark the videos in Figure 3.

Q2.6

C_T = 0 means the forecast commutes with T. It does not mean the forecast is right. Blank and copy-last commute for a structural reason: a black frame is still black after a mirror or a rotation, and repeating the last frame commutes with a pixel permutation. Their transformed error equals their original error, 0.0426 and 0.0641, so they are consistent and wrong. E_T has to sit next to C_T. Otherwise a constant predictor looks perfect.

Q2.7

Video 9168, the largest rotation gap for seed 0 (C_R = 0.00572). Rows 3 and 4 of that panel share the rotated input. Row 4 is the original forecast after the same rotation: both digits are still there, blurred, and they keep moving apart. Row 3 is a fresh forecast from the rotated input: one digit fades within the five horizons and the other smears into a block. The model's account of the motion changes. Rotating a forecast it was willing to make on the upright clip does not match the forecast it makes once the clip is rotated. Shown the rotated clip, it loses a digit instead of continuing both trajectories.

Q2.8

Yes. Mean C_R is 0.00293 against mean C_M of 0.00174, and E_R is 0.0239 against E_M of 0.0229 and an original E(5) of 0.0223. Figure 4 sits further to the right for rotation than for mirror. From Q2.1, a 90° rotation moves more of the direction histogram than a mirror does. From Q2.2, it also changes the shape of a digit more. Both of those are about appearance and about which directions were common, not about a different law of motion. The transformed errors stay far below the blank predictor (0.0426). The model is still forecasting. The speed test, which changes the step and not the appearance, is the one that reaches the blank error. That is a different failure.

Table 4. Faster motion

Model	E(2) at 1×	E(2) at 2×	E(2) at 3×	E(5) at 1×	E(5) at 2×
A	0.0188 (0.0191, 0.0184)	0.0449 (0.0437, 0.0461)	0.0523 (0.0512, 0.0535)	0.0223 (0.0227, 0.0218)	0.0497 (0.0474, 0.0519)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

Table 4. Speed comparison on the test set at t = 1. E(5) is not defined at 3×, because only two target frames exist. Blank's E(2) is 0.0426, 0.0425 and 0.0426 at 1×, 2× and 3×.

Figure 5. Five input frames, the first two predictions, and the first two true futures at 1×, 2× and 3×.

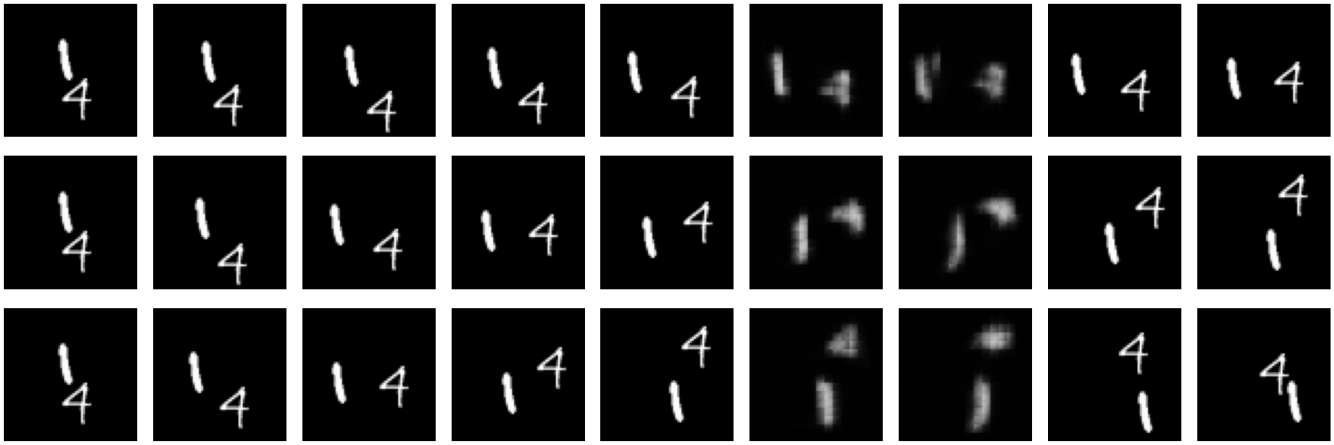


Figure 5. Seed-0 model A on test video 9021, chosen so the displayed true frames do not overlap. Each row is one speed: five input frames, the first two predictions, and the first two true future frames, at 1×, 2× and 3×.

Q2.9

Model A falls apart at 2×, and 3× is worse. E(2) goes from 0.0188 at 1× to 0.0449 at 2× and 0.0523 at 3×. Blank's E(2) is about 0.0426, so at 2× the forecast is already no better than predicting black. This is earlier than the median argument in Q2.5. Twice the median speed is 6.63 pixels, inside a reach of 8, and three times the median is 9.95, outside it. What that estimate missed is the tail and the width of a digit. Twice the 90th percentile is 8.25 pixels, already past the axis-aligned reach, so a real share of 2× motion cannot be joined to the previous position in one update. A digit is also wider than the one pixel whose flow was measured. Figure 5, video 9021, shows the failure as smear. At 1× the 1 and the 4 stay sharp enough to follow. At 2× and 3× the forecast is a soft ghost near the last input position, while the true digits have moved on. A stall would repeat a sharp digit. A duplicate would show two copies. A vanishing digit would go to black. This is the smear: the model no longer has a position to commit to.

Q2.10

No. Model A never trained on k = 2 or k = 3, and 2× motion also sits on the edge of the 8-pixel reach. Either fact can produce Figure 5. The split is model C, the same architecture trained with k drawn from {1, 2, 3}, against model D, a larger reach trained on the same speeds. If C repairs 2× and still fails at 3×, then 2× was missing from the training distribution and 3× is beyond the architecture. If C fails at 2× as well, the reach was already the limit, and more fast clips cannot move information further than the convolutions allow.

Q2.11

Q2.11 (written before training models B, C, and D)

Model B should remove the mirror and rotation failures. Those errors stay well below the blank predictor: transformed E(5) is only a little higher than the original E(5), so this looks like a missing symmetry rather than a broken motion rule, and B is the only one of these three models that sees g drawn from G. Model C should be the one that repairs 2×, because twice the median speed is 6.6 pixels, inside the 8-pixel reach, and C is the first model that actually sees k = 2; it should not repair 3×, where a step of about 10 pixels is beyond that reach and E(2) is already worse than blank. Model D keeps C's speed augmentation and extends the reach to 16 pixels, so it is the model that should bring 3× back, while the mirror and rotation gaps stay until the forecaster is symmetrised.

Q2.12

Model D is model A's ConvLSTM with one change. Layer 1 stays a 3×3 with dilation 1. Layer 2 is a 3×3 with dilation 3 and padding 3, so it reads offsets −3, 0 and +3. Dilation adds no weights, so

the parameter count stays 480,784, under the limit of 721,176. There is no attention and no extra layer. The 9-step rollout, the shapes, the loss and the rest of the training protocol are unchanged. The only protocol change is the one in the model table: k is drawn from {1, 2, 3}. Reach uses the same rules as Q2.3. One cell is 4 original pixels. Layer 1 adds a radius of 1 cell and layer 2 adds a radius of 3. The longest path, the new input or layer 1's previous state through both convolutions, has Chebyshev radius 4 cells, which is 16 original pixels. The summed offsets are every integer from −4 to 4, so that neighbourhood has no holes. Layer 2's own previous state only reaches 12 pixels, which is the shorter path. 16 is strictly larger than model A's 8. Three times the median speed, 9.95, and three times the 90th percentile, about 12.4, are inside 16 and outside 8.

Table 5. What changes with the data, and with the reach

Model	E(5)	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	Params	Train s	Infer s
A	0.0223 (0.0227, 0.0218)	0.00174 (0.00145, 0.00203)	0.0229 (0.0232, 0.0225)	0.00293 (0.00237, 0.00349)	0.0239 (0.0241, 0.0238)	0.0188 (0.0191, 0.0184)	0.0449 (0.0437, 0.0461)	0.0523 (0.0512, 0.0535)	480,784	11.9 (11.9, 11.9)	0.42 (0.421, 0.42)
B	0.0228 (0.0236, 0.0221)	9.82e-4 (0.00101, 9.58e-4)	0.0228 (0.0236, 0.0221)	0.00125 (0.00125, 0.00125)	0.0228 (0.0235, 0.0221)	0.0191 (0.0196, 0.0186)	0.0425 (0.0419, 0.0432)	0.048 (0.0467, 0.0493)	480,784	12.5 (12.5, 12.5)	0.422 (0.423, 0.422)
C	0.027 (0.0278, 0.0262)	0.00136 (0.00145, 0.00127)	0.0272 (0.0279, 0.0265)	0.0026 (0.0022, 0.00299)	0.0282 (0.0287, 0.0276)	0.0233 (0.0241, 0.0224)	0.0288 (0.0295, 0.0281)	0.0336 (0.0339, 0.0332)	480,784	11.8 (11.8, 11.8)	0.423 (0.422, 0.424)
D	0.0305 (0.0311, 0.0299)	3.36e-4 (2.45e-4, 4.28e-4)	0.0305 (0.0311, 0.0299)	7.90e-4 (6.45e-4, 9.35e-4)	0.0308 (0.0314, 0.0303)	0.028 (0.0285, 0.0275)	0.0311 (0.0316, 0.0307)	0.0342 (0.0345, 0.034)	480,784	16.1 (16.1, 16.1)	0.613 (0.614, 0.613)
D+B	0.0302 (0.0308, 0.0295)	2.44e-4 (1.51e-4, 3.38e-4)	0.0302 (0.0308, 0.0296)	2.90e-4 (1.95e-4, 3.86e-4)	0.0302 (0.0308, 0.0296)	0.0277 (0.0282, 0.0273)	0.0311 (0.0315, 0.0307)	0.034 (0.0344, 0.0336)	480,784	16.6 (16.6, 16.6)	0.613 (0.613, 0.612)

Table 5. Test metrics for the learned models. D+B is model D trained with both speed and symmetry augmentation. Train s is the mean seconds per epoch. Infer s is one forward pass of the 1,000 test videos after the batch is on the Tesla T4. Each learned entry is mean, then seed 0 and seed 1.

Q2.13

The mirror and rotation gaps are an appearance failure, in the sense of Q2.2: the motion law is unchanged and the digits are not. Model B, which only adds g drawn from G, cuts C_M from 0.00174 (0.00145, 0.00203) to 9.82e-4 (0.00101, 9.58e-4) and C_R from 0.00293 (0.00237, 0.00349) to 0.00125 (0.00125, 0.00125). Its transformed errors fall into line with its own E(5) of 0.0228 (0.0236, 0.0221). Model C, which changes speed and not symmetry, leaves C_R at 0.0026 (0.0022, 0.00299), close to model A. So the symmetry gap moves when the training distribution is symmetrised, and it does not move when only the speeds change. The speed failure is unfamiliar motion, not architectural reach. Model C has the same 8-pixel reach as A and cuts E(2) at 2× from 0.0449 (0.0437, 0.0461) to 0.0288 (0.0295, 0.0281) and at 3× from 0.0523 (0.0512, 0.0535) to 0.0336 (0.0339, 0.0332). Both of those are better than blank, which sits near 0.0426. Model D, with reach 16 and the same speed augmentation, does not beat C: its 3× E(2) is 0.0342 (0.0345, 0.034). The conditional in the question does not hold. C is not failing at 3× while D succeeds. More examples of fast motion were enough, inside the original reach. That does not mean data can extend a reach. A 3×3 kernel has a zero weight on every pixel outside its offsets, for every value of the weights. Training changes those weights. It does not change which pixels are read. D was the test of that limit, and on this data the limit was not what separated the errors.

Q2.14

Yes. Original E(5) goes from 0.0223 (0.0227, 0.0218) for A to 0.0228 (0.0236, 0.0221) for B, 0.027 (0.0278, 0.0262) for C, 0.0305 (0.0311, 0.0299) for D and 0.0302 (0.0308, 0.0295) for D+B. B’s cost is small beside the drop in the consistency gaps, and it is worth it. C’s cost is larger, and it is the cost of no longer collapsing to the blank predictor at 2× and 3×. That trade is worth it for the question being asked. D’s further cost over C

bought no speed improvement. The dilation was not worth it on these clips. D+B matches D on the original clips and is the better of those two under mirror and rotation, which is why it is the one wrapped below.

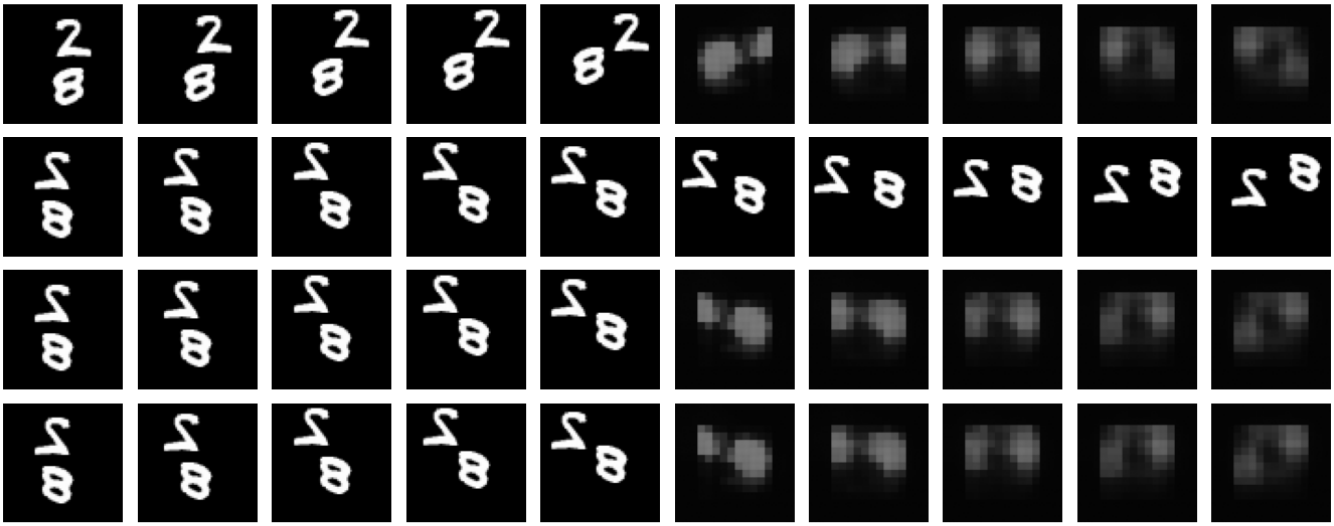
Table 6. Symmetrised forecasters

For any g in G and any T in {M, R}, right-multiplication by T is a bijection of G. Set h = g T, so g^{−1} = T h^{−1}. Then f̄(TX) = (1/8) Σ_g g^{−1} f(g T X) = (1/8) Σ_h T(h^{−1} f(h X)) = T(f̄(X)), because T permutes pixels and therefore passes through the average. So f̄(TX) = T(f̄(X)), and C_M = C_R = 0 for every underlying f. The values below are the floating-point remainder.

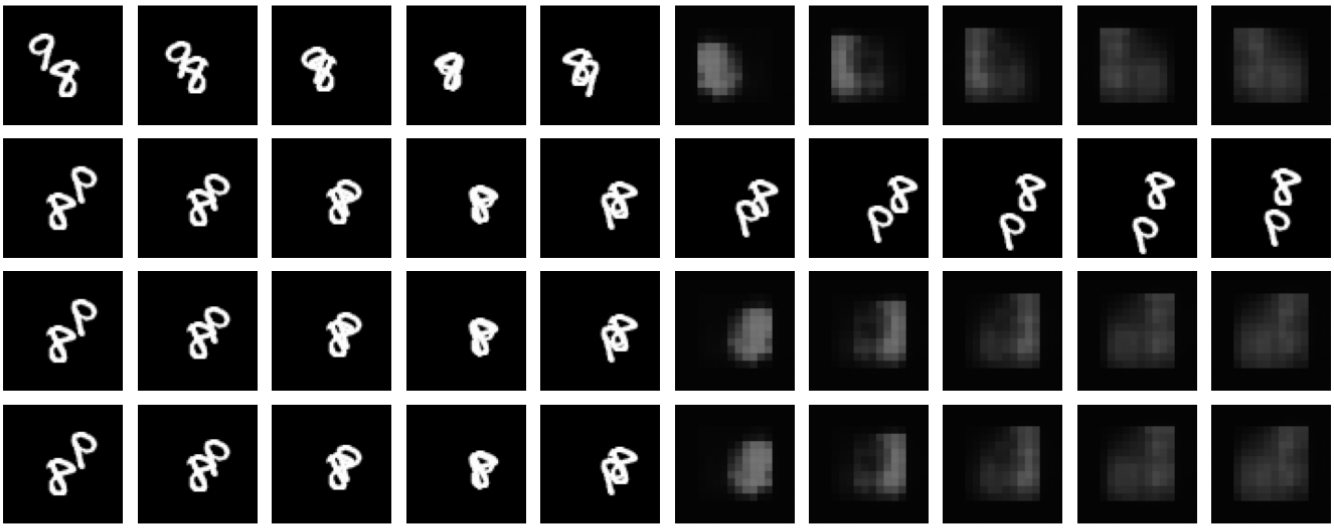
Model	E(5)	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	Params	Train s	Infer s
sym B	0.0223 (0.023, 0.0216)	7.21e-17 (7.09e-17, 7.33e-17)	0.0223 (0.023, 0.0216)	5.82e-17 (5.74e-17, 5.90e-17)	0.0223 (0.023, 0.0216)	0.0187 (0.0192, 0.0182)	0.0417 (0.0413, 0.0422)	0.0469 (0.046, 0.0477)	480,784	12.5 (12.5, 12.5)	3.43 (3.42, 3.44)
sym D	0.0304 (0.031, 0.0298)	5.06e-17 (5.10e-17, 5.02e-17)	0.0304 (0.031, 0.0298)	3.84e-17 (3.92e-17, 3.75e-17)	0.0304 (0.031, 0.0298)	0.0279 (0.0285, 0.0274)	0.0312 (0.0316, 0.0307)	0.0342 (0.0345, 0.0339)	480,784	16.1 (16.1, 16.1)	4.93 (4.92, 4.94)
sym D+B (final)	0.0301 (0.0308, 0.0294)	4.37e-17 (3.81e-17, 4.93e-17)	0.0301 (0.0308, 0.0294)	3.31e-17 (2.89e-17, 3.72e-17)	0.0301 (0.0308, 0.0294)	0.0276 (0.0281, 0.0271)	0.031 (0.0315, 0.0305)	0.0339 (0.0343, 0.0335)	480,784	16.6 (16.6, 16.6)	4.94 (4.93, 4.95)

Table 6. The same columns as Table 5, for the symmetrised forecasters. Train s is the underlying learned model. Symmetrisation is not trained. C_M and C_R are floating-point zeros and are written in scientific notation. The final model is symmetrised D+B, not symmetrised D. D+B's original E(5) stays within 5% of D's, and both of its transformed errors are lower, so Table 5 supports wrapping D+B.

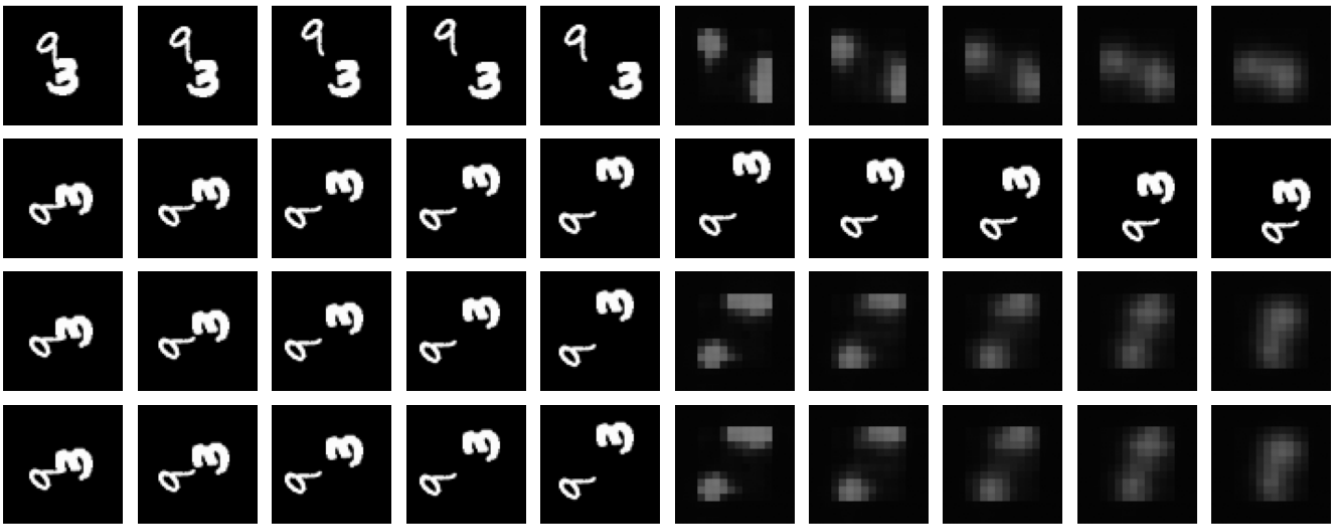
DB mirror video 9642



DB mirror video 9279



DB rotation video 9168



DB rotation video 9925

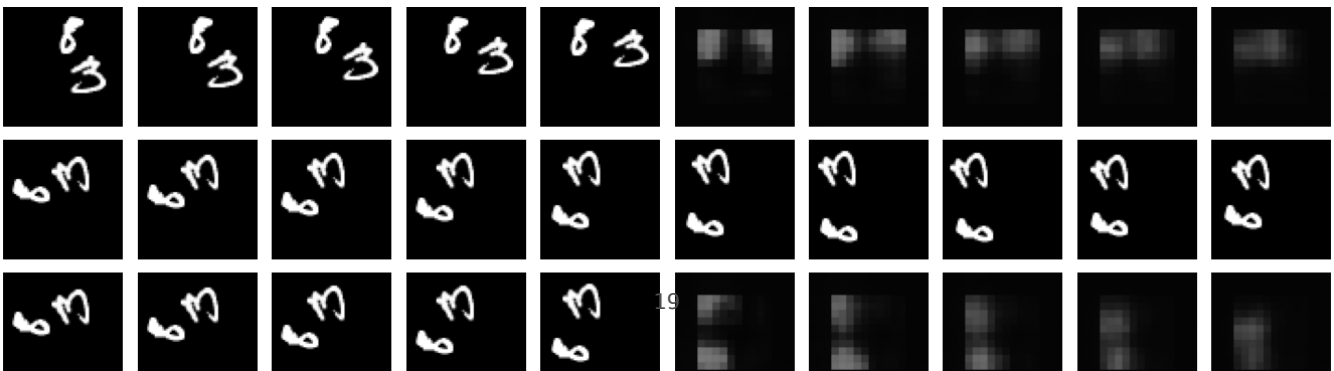


Figure 6(a). The final model, symmetrised D+B, seed 0, on the same videos as Figure 3: mirror 9642 and 9279, rotation 9168 and 9925. Rows match Figure 3. GIFs: Question2/outputs/gifs/figure6_DB_mirror_0.gif, figure6_DB_mirror_1.gif, figure6_DB_rotation_0.gif, figure6_DB_rotation_1.gif.

Figure 5. Five input frames, the first two predictions, and the first two true futures at 1×, 2× and 3×.

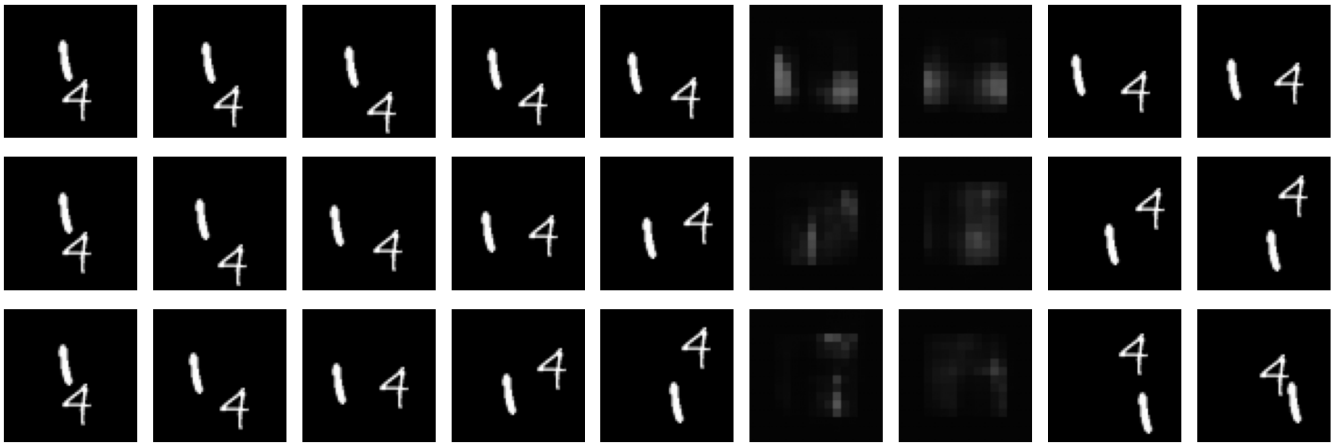


Figure 6(b). The same final model on test video 9021, the video in Figure 5, at 1×, 2× and 3×.

Q2.15

Learned symmetry is model B. Its C_M is 9.82e-4 (0.00101, 9.58e-4) and its C_R is 0.00125 (0.00125, 0.00125), against 0.00174 (0.00145, 0.00203) and 0.00293 (0.00237, 0.00349) for A. Its E_M and E_R match its E(5) of 0.0228 (0.0236, 0.0221). The gap shrinks. It does not become zero. Inference stays one forward pass, 0.422 (0.423, 0.422) s. Built-in symmetry is the average over G. It is the exact one, because the short argument above does not depend on f. Wrapped B has C_M = 7.21e-17 (7.09e-17, 7.33e-17) and C_R = 5.82e-17 (5.74e-17, 5.90e-17). The cost is the eight passes: 3.43 (3.42, 3.44) s against 0.422 (0.423, 0.422) s, and 4.94 (4.93, 4.95) s for the final model against 0.613 (0.613, 0.612) s for D+B unwrapped. Averaging eight forecasts blurs them when they disagree about where a digit is. Here they mostly agree. Wrapped B's E(5) is 0.0223 (0.023, 0.0216), no worse than learned B, and wrapped D+B's E(5) is 0.0301 (0.0308, 0.0294), the same as D+B to three figures. Built-in symmetry did not hurt accuracy on this test set. It would hurt a model whose eight transformed forecasts pointed different ways.

Q2.16

On video 9168 the disagreement is gone and the forecast is not repaired. In Figure 3, row 4 kept two blurred digits and row 3 lost one. In Figure 6(a) those two rows match: both are the same coarse, blocky field. That match is the group average. It is why C_R is a floating-point zero. It is not a better account of the bounce. The upright forecast in row 1 is blocky as well, which lines up with the original E(5) rising from 0.0223 for A to about 0.030 for D+B. The part of the failure that was a broken symmetry was fixed by the wrapper, and by D+B's symmetry augmentation before that. The part that was a lost digit was not turned back into two sharp trajectories. Figure 6(b) is the same story at speed. The error at 3× is lower than model A's, and the frames are still smears rather than a 1 and a 4 at the true positions.

Q2.17

The final model has not learned a motion rule that stands without help. Model C, with A's 8-pixel reach, cuts E(2) at 2× from 0.0449 to 0.0288 and at 3× from 0.0523 to 0.0336, so the speed collapse was motion the training clips never showed. D's 16-pixel reach does not beat C, so reach was not the binding limit. Mirror and rotation consistency are exact only because the final forecaster averages all eight symmetries. That is imposed. Model B learns a smaller gap, not a zero gap. A skeptic should ask for a change that neither the training distribution nor the wrapper covers. The test it has not passed is a rotation by an angle other than a multiple of 90°, or a speed above 3×. Figure 6 is the other warning: the error can fall while the digits turn into blocks.

AI use

This assignment was completed with Cursor’s coding agent (Grok 4.7). The prompts were the assignment PDF, the instruction to finish both questions without further questions, the instruction to run Question 2 on Kaggle, and a Kaggle API token used only to authenticate the Kaggle CLI. The agent wrote the ConvLSTM code, the checks, the training and evaluation scripts, the Question 1 design, the figures, and this report. Training ran on Kaggle. The paragraphs under Q2.5 and Q2.11 are the files committed before those experiments and were not edited afterwards. Numbers in the tables were taken from the saved score files. The architecture diagram was drawn for this design. The Moving MNIST figures were produced by the evaluation code.